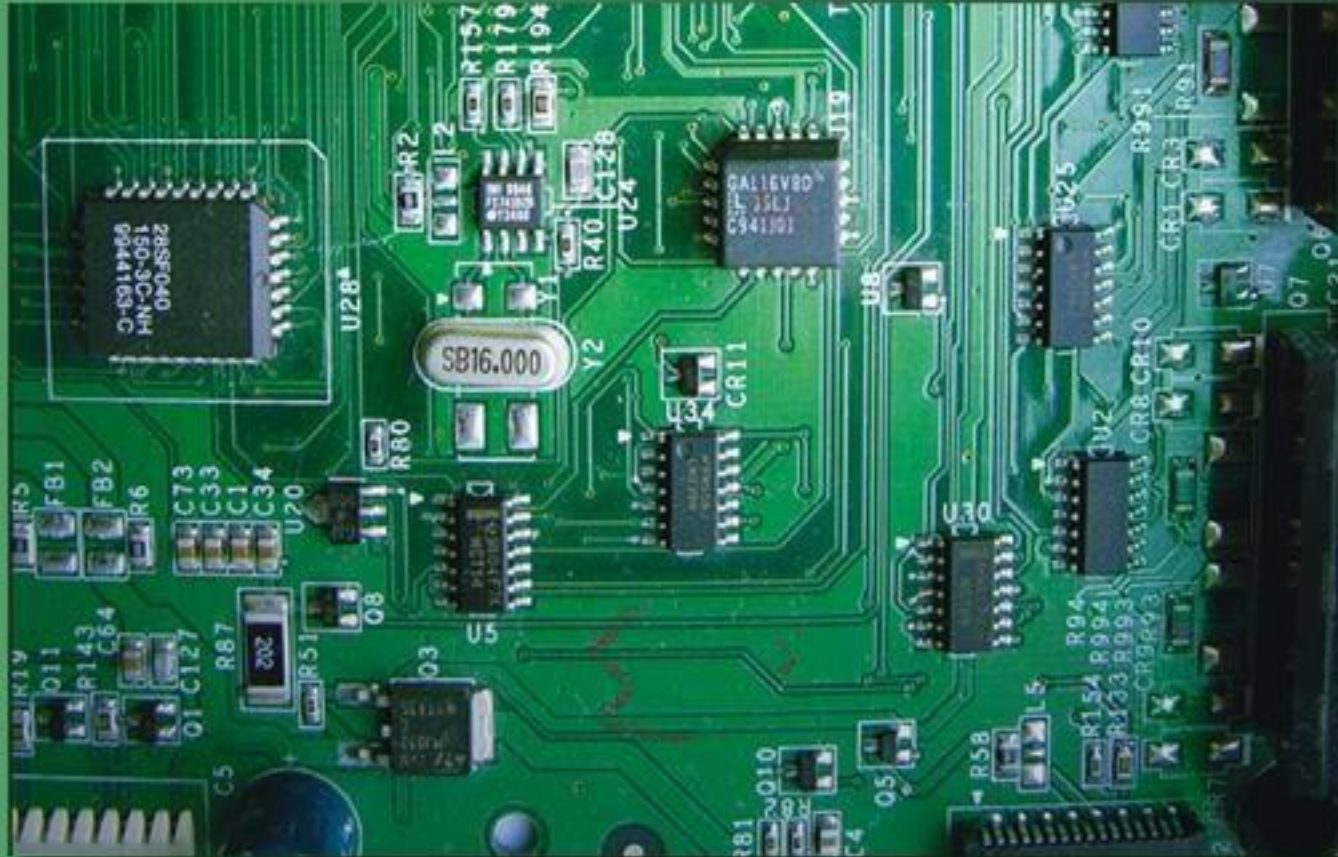


The Intel Microprocessors

8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium 4, and Core2 with 64-bit Extensions

Architecture, Programming, and Interfacing



EIGHTH EDITION

Barry B. Brey

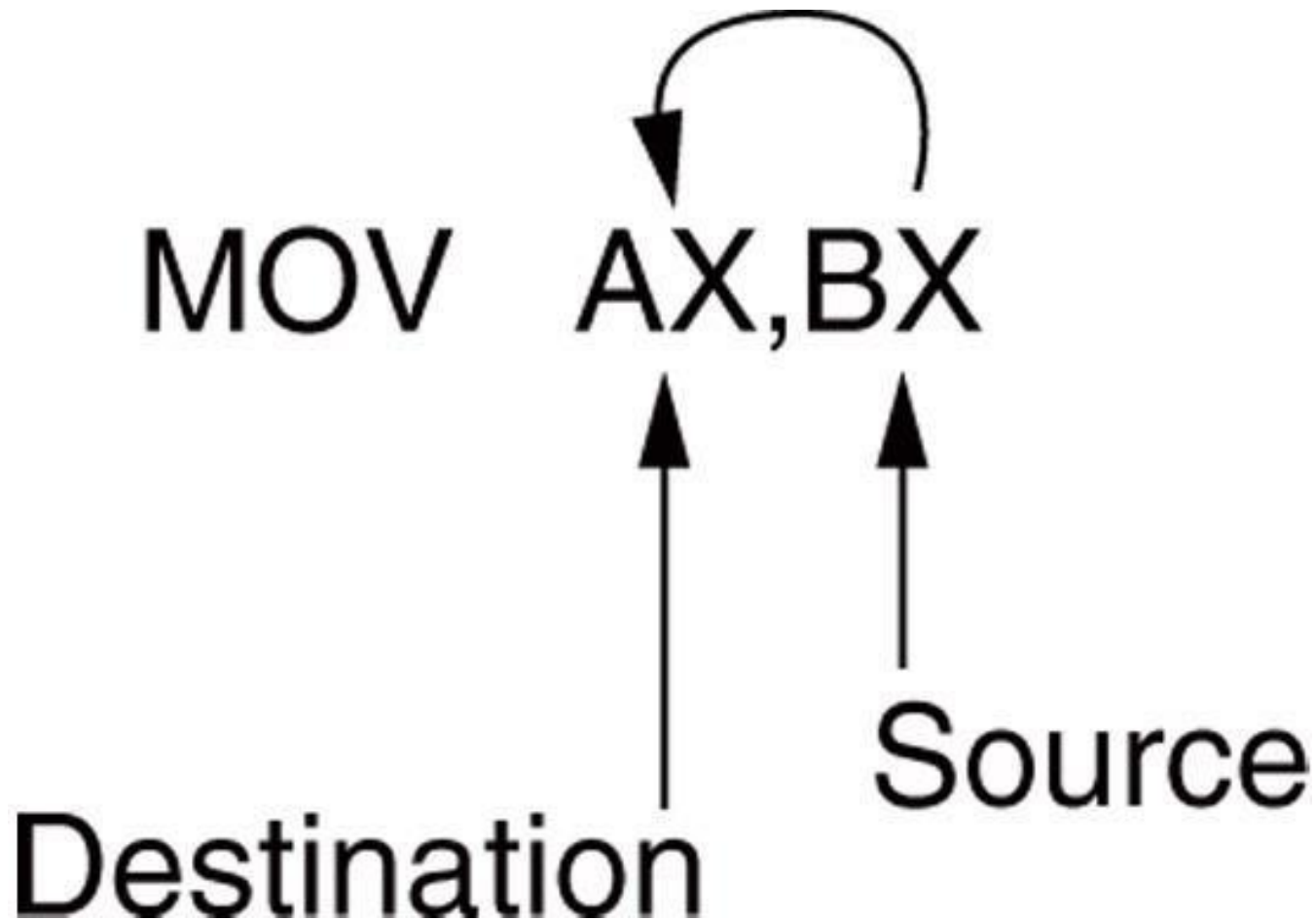
PEARSON

Chapter 3: Addressing Modes

3–1 DATA ADDRESSING MODES

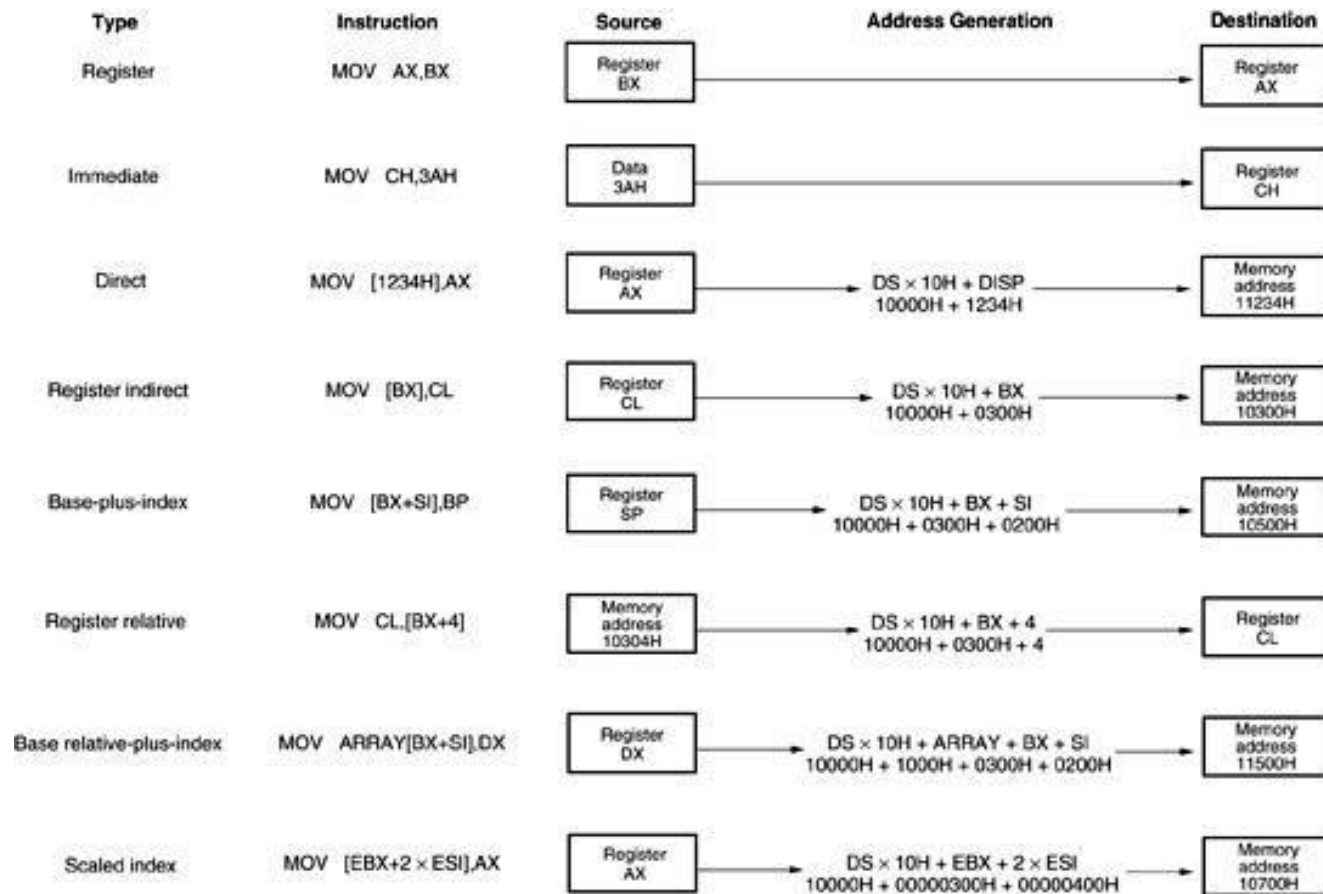
- MOV instruction is a common and flexible instruction.
 - provides a basis for explanation of data-addressing modes
- Figure 3–1 illustrates the MOV instruction and defines the direction of data flow.
- **Source** is to the right and **destination** the left, next to the opcode MOV.
 - an **opcode**, or operation code, tells the microprocessor which operation to perform

Figure 3–1 The MOV instruction showing the source, destination, and direction of data flow.



- Figure 3–2 shows all possible variations of the data-addressing modes using MOV.
- These data-addressing modes are found with all versions of the Intel microprocessor.
 - except for the scaled-index-addressing mode, found only in 80386 through Core2

Figure 3–2 8086–Core2 data-addressing modes.



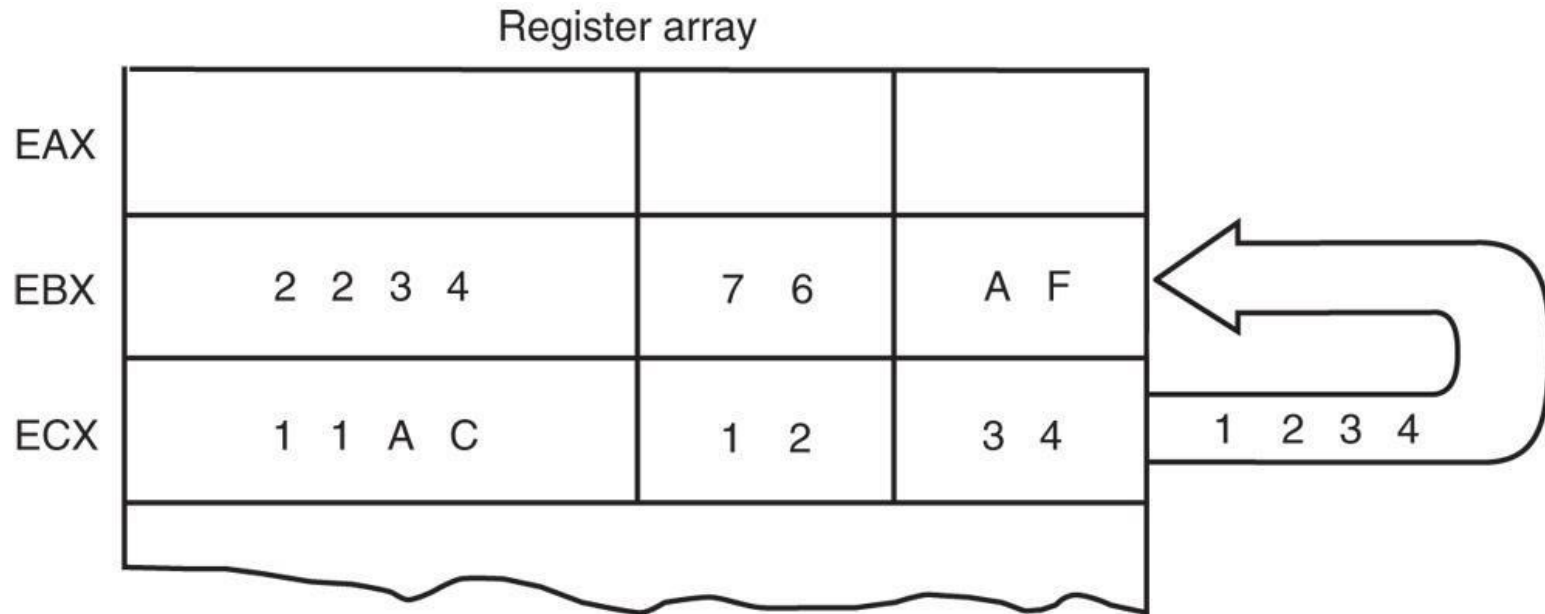
Notes: EBX = 00000300H, ESI = 00000200H, ARRAY = 1000H, and DS = 1000H

Register Addressing

- The most common form of data addressing.
 - once register names learned, easiest to apply.
- The microprocessor contains these 8-bit register names used with register addressing: AH, AL, BH, BL, CH, CL, DH, and DL.
- 16-bit register names: AX, BX, CX, DX, SP, BP, SI, and DI.

- In 80386 & above, extended 32-bit register names are: EAX, EBX, ECX, EDX, ESP, EBP, EDI, and ESI.
- 64-bit mode register names are: RAX, RBX, RCX, RDX, RSP, RBP, RDI, RSI, and R8 through R15.
- Important for instructions to use registers that are the same size.
 - *never* mix an 8-bit \with a 16-bit register, an 8- or a 16-bit register with a 32-bit register
 - this is not allowed by the microprocessor and results in an error when assembled

Figure 3–3 The effect of executing the MOV BX, CX instruction at the point just before the BX register changes. Note that only the rightmost 16 bits of register EBX change.

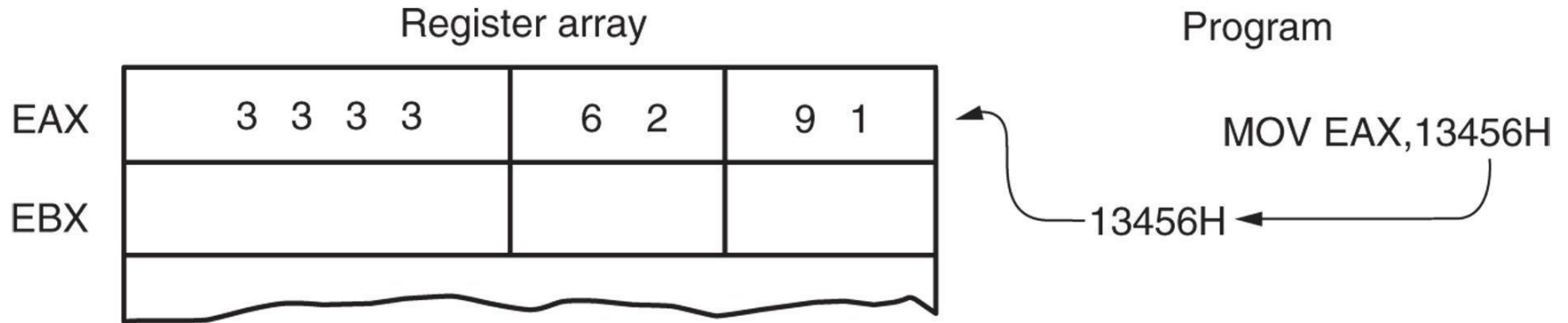


- Figure 3–3 shows the operation of the MOV BX, CX instruction.
- The source register's contents do not change.
 - the destination register's contents do change
- The contents of the destination register or destination memory location change for all instructions except the CMP and TEST instructions.
- The MOV BX, CX instruction does not affect the leftmost 16 bits of register EBX.

Immediate Addressing

- Term *immediate* implies that data immediately follow the hexadecimal opcode in the memory.
 - immediate data are constant data
 - data transferred from a register or memory location are variable data
- Immediate addressing operates upon a byte or word of data.
- Figure 3–4 shows the operation of a MOV EAX,13456H instruction.

Figure 3–4 The operation of the MOV EAX,3456H instruction. This instruction copies the immediate data (13456H) into EAX.



- As with the MOV instruction illustrated in Figure 3–3, the source data overwrites the destination data.

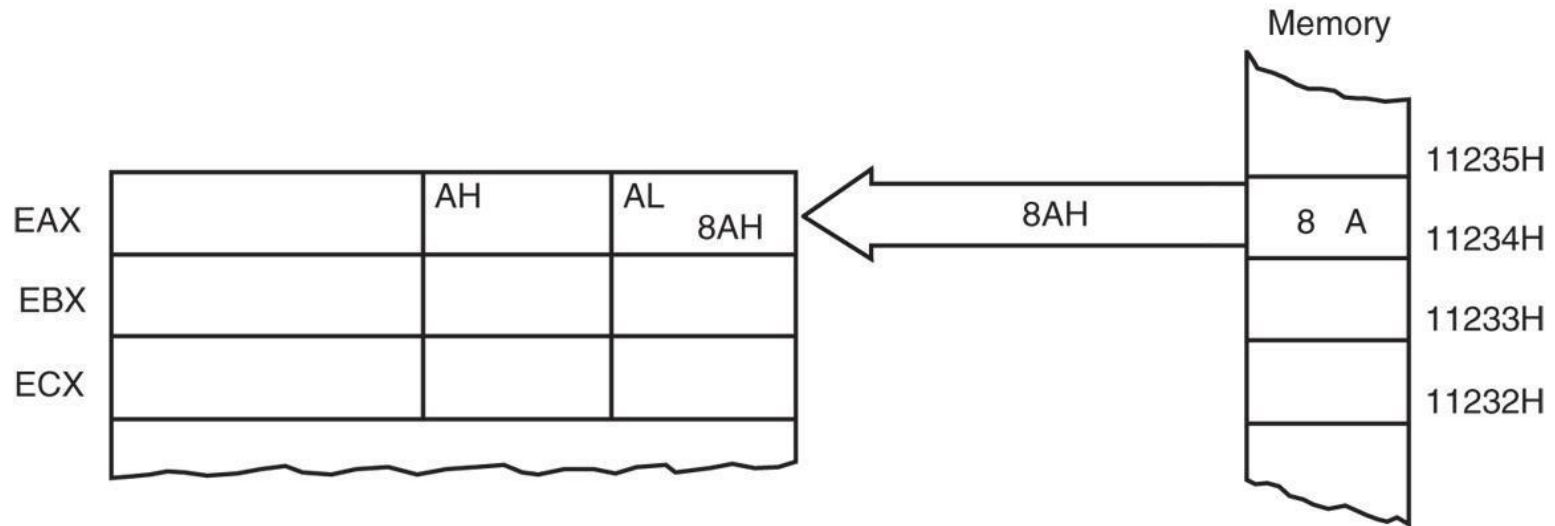
Direct Data Addressing

- Applied to many instructions in a typical program.
- Two basic forms of direct data addressing:
 - direct addressing, which applies to a MOV between a memory location and AL, AX, or EAX
 - displacement addressing, which applies to almost any instruction in the instruction set
- Address is formed by adding the displacement to the default data segment address or an alternate segment address.

Direct Addressing

- Direct addressing with a MOV instruction transfers data between a memory location, located within the data segment, and the AL (8-bit), AX (16-bit), or EAX (32-bit) register.
 - usually a 3-byte long instruction
- MOV AL,DATA loads AL from the data segment memory location DATA(1234H).
 - DATA is a symbolic memory location, while 1234H is the actual hexadecimal location

Figure 3–5 The operation of the MOV AL,[1234H] instruction when DS=1000H .

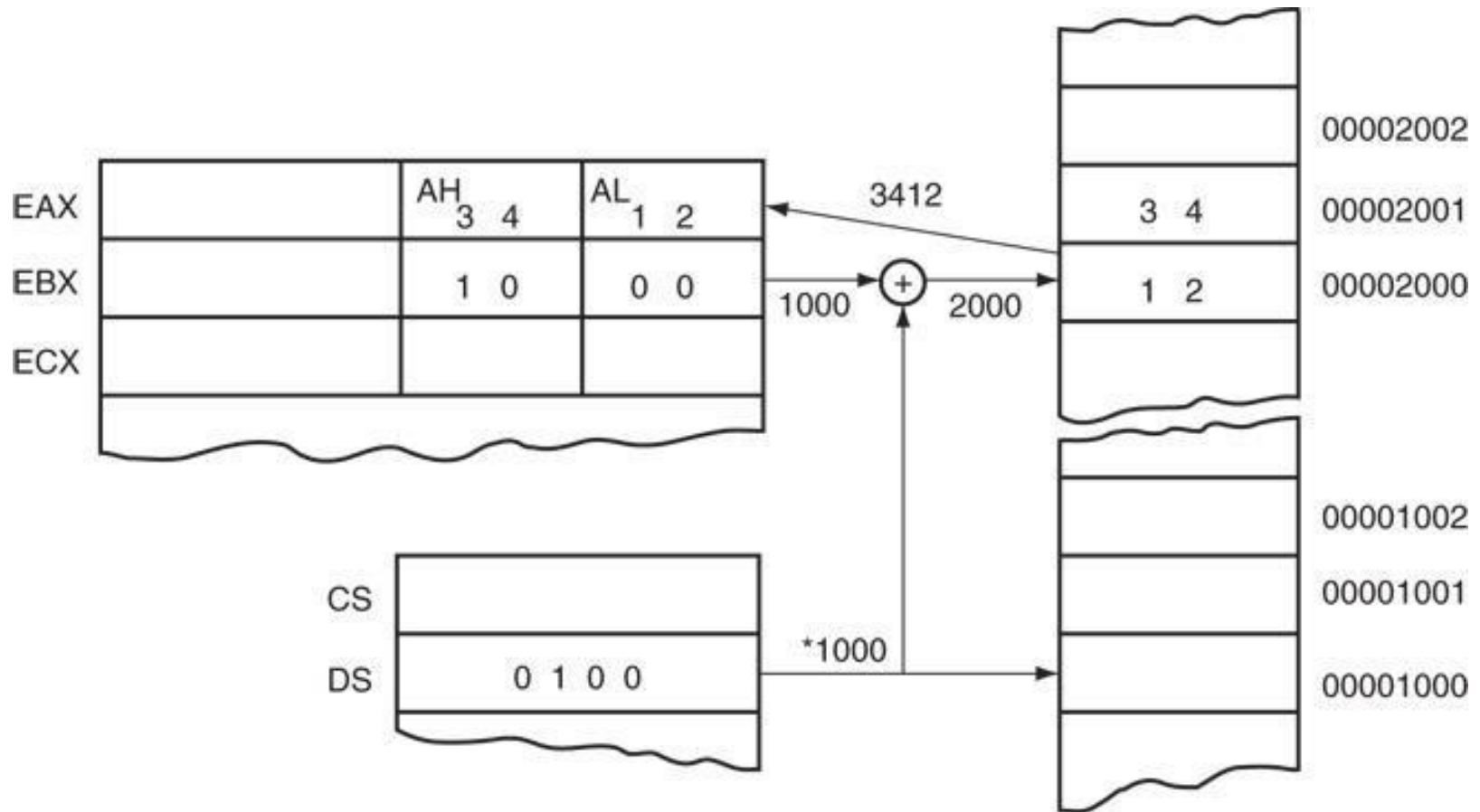


- This instruction transfers a copy contents of memory location 11234H into AL.
 - the effective address is formed by adding 1234H (the offset address) and 10000H (the data segment address of 1000H times 10H) in a system operating in the real mode

Register Indirect Addressing

- Allows data to be addressed at any memory location through an offset address held in any of the following registers: BP, BX, DI, and SI.
- In addition, 80386 and above allow register indirect addressing with any extended register except ESP.
- In the 64-bit mode, the segment registers serve no purpose in addressing a location in the flat model.

Figure 3–6 The operation of the MOV AX,[BX] instruction when BX = 1000H and DS = 0100H. Note that this instruction is shown after the contents of memory are transferred to AX.



*After DS is appended with a 0.

- The **data segment** is used by default with register indirect addressing or any other mode that uses BX, DI, or SI to address memory.
- If the BP register addresses memory, the **stack segment** is used by default.
 - these settings are considered the default for these four index and base registers
- For the 80386 and above, EBP addresses memory in the stack segment by default.
- EAX, EBX, ECX, EDX, EDI, and ESI address memory in the data segment by default.

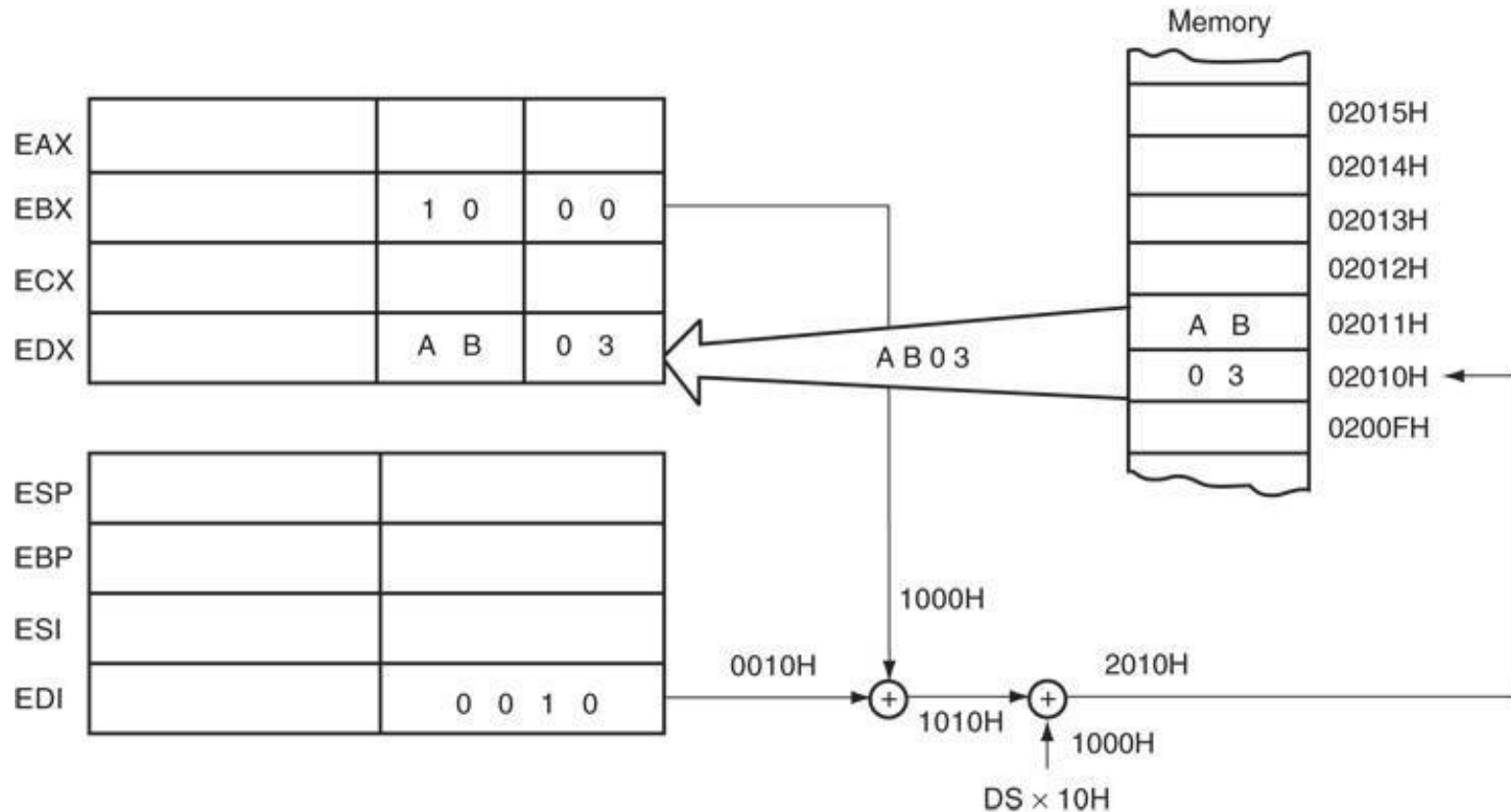
Base-Plus-Index Addressing

- Similar to indirect addressing because it indirectly addresses memory data.
- The base register often holds the beginning location of a memory array.
 - the index register holds the relative position of an element in the array
 - whenever BP addresses memory data, both the stack segment register and BP generate the effective address

Locating Data with Base-Plus-Index Addressing

- Figure 3–8 shows how data are addressed by the MOV DX,[BX + DI] instruction when the microprocessor operates in the real mode.
- The Intel assembler requires this addressing mode appear as [BX][DI] instead of [BX + DI].
- The MOV DX,[BX + DI] instruction is MOV DX,[BX][DI] for a program written for the Intel ASM assembler.

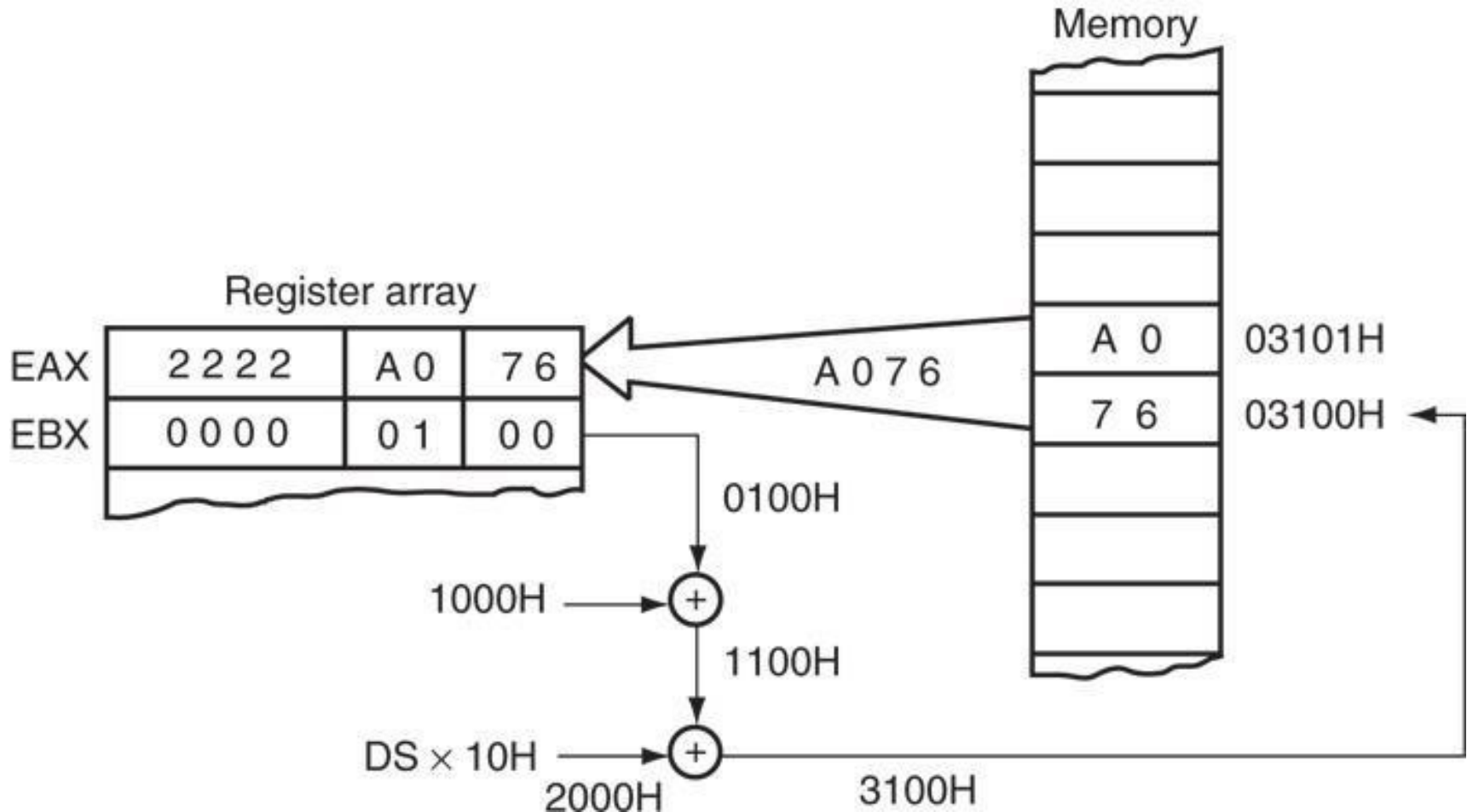
Figure 3–8 An example showing how the base-plus-index addressing mode functions for the MOV DX,[BX + DI] instruction. Notice that memory address 02010H is accessed because DS=0100H, BX=1000H and DI=0010H.



Register Relative Addressing

- Similar to base-plus-index addressing and displacement addressing.
 - data in a segment of memory are addressed by adding the displacement to the contents of a base or an index register (BP, BX, DI, or SI)
- Figure 3–10 shows the operation of the MOV AX,[BX+1000H] instruction.
- A real mode segment is 64K bytes long.

Figure 3–10 The operation of the MOV AX, [BX+1000H] instruction, when BX=0100H and DS=0200H .



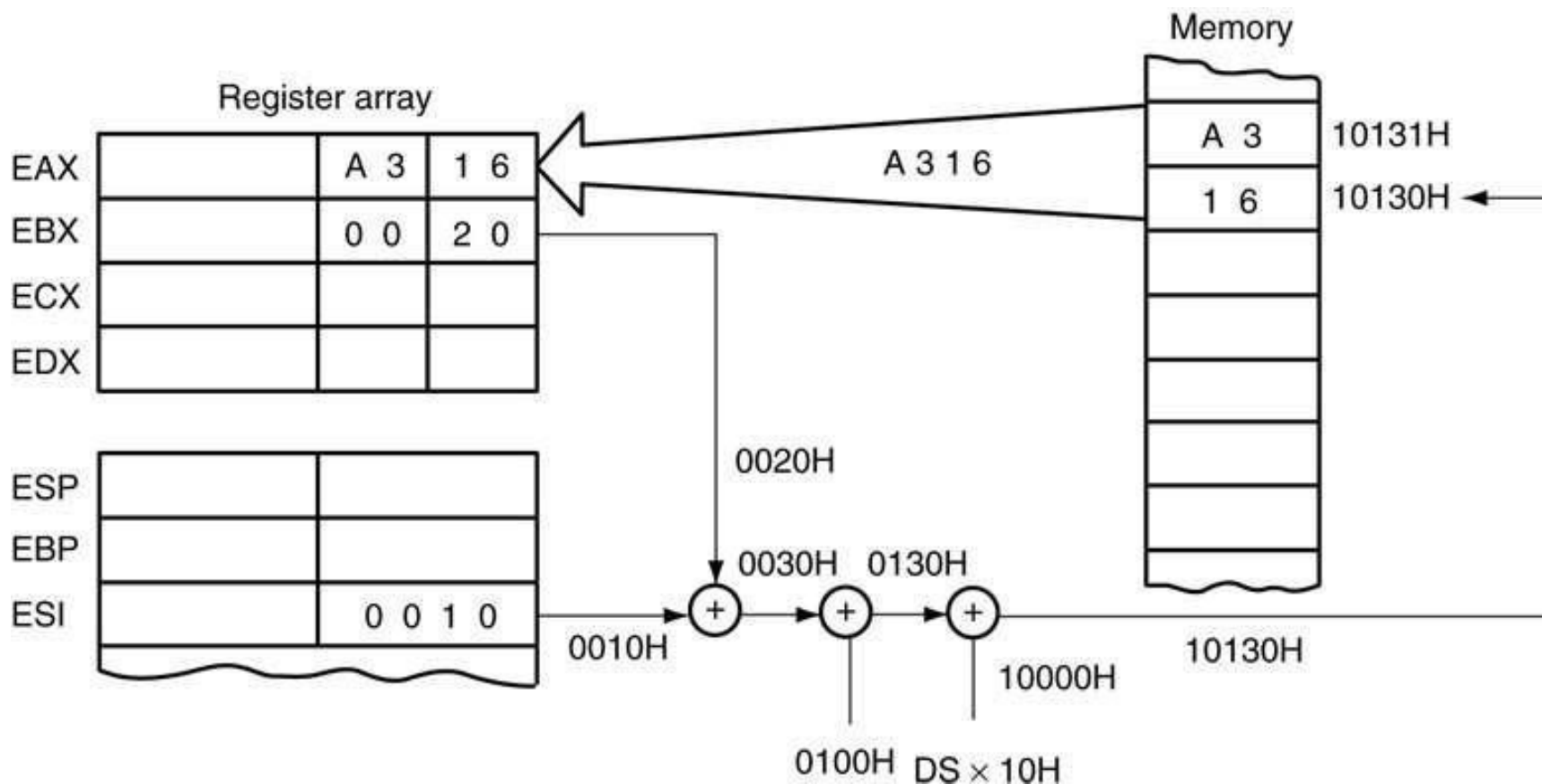
Base Relative-Plus-Index Addressing

- Similar to base-plus-index addressing.
 - adds a displacement
 - uses a base register and an index register to form the memory address
- This type of addressing mode often addresses a two-dimensional array of memory data.

Addressing Data with Base Relative-Plus-Index

- Least-used addressing mode.
- Figure 3–12 shows how data are referenced if the instruction executed by the microprocessor is `MOV AX,[BX + SI + 100H]`.
 - displacement of 100H adds to BX and SI to form the offset address within the data segment
- This addressing mode is too complex for frequent use in programming.

Figure 3–12 An example of base relative-plus-index addressing using a MOV AX,[BX+SI+0100H] instruction. Note: DS=1000H



Scaled-Index Addressing

- Unique to 80386 - Core2 microprocessors.
 - uses two 32-bit registers (a base register and an index register) to access the memory
- The second register (index) is multiplied by a scaling factor.
 - the scaling factor can be 1x, 2x, 4x, 8x
- A scaling factor of 1 is implied and need not be included in the assembly language instruction (MOV AL,[EBX + ECX]).